Theme 01

Razorpay Buildathon

PROJECT TITLE

AEGIS — Autonomous Commerce Operating System

ONE-LINE PITCH

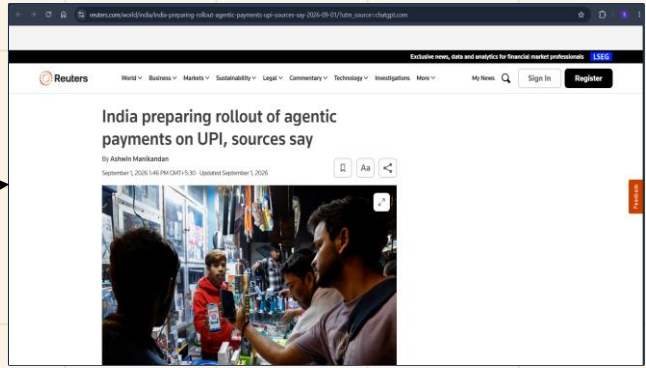
AEGIS *lets AI buyers grow and complete commerce autonomously—while every money action is explainable, cryptographically authorized, deterministically gated, and auditably executed.*

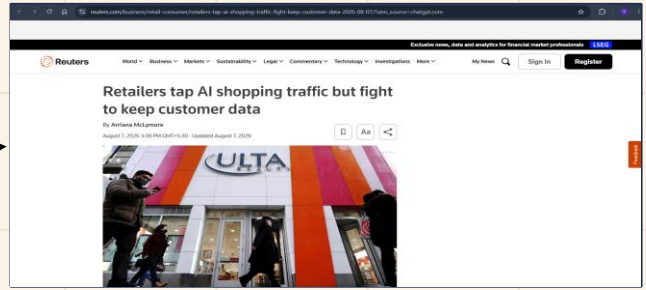
Theme 01: AI Growth & Agentic Commerce

Sanjeev Kumar S
sanjeevkumaredu8@gmail.com

Problem Definition

News article of problem faced





AI Buyers Are Becoming Autonomous — Commerce Infrastructure Isn't Ready

- AI can discover and recommend products, but payment execution remains a high-risk boundary.
- Traditional commerce systems assume humans make the final purchase decision.
- Autonomous agents introduce new attack surfaces:
  - Prompt Injection
  - Amount Escalation
  - Merchant / Recipient Substitution
  - Category Manipulation
  - Intent & Parameter Tampering
- Existing agentic workflows can create a dangerous gap between what the AI intended and what actually gets paid.

The Core Problem

How do we let AI buyers act autonomously without giving them unrestricted financial authority?

Missing Layer

AI Intelligence → Direct Payment Access

What's needed:

AI Decision → Cryptographic Authorization → Deterministic Policy → Payment

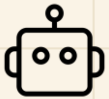
Proposed Solution – AEGIS

AEGIS is a deterministic trust boundary for autonomous commerce agents, mediating every purchase action before it reaches payment infrastructure. It separates AI-driven discovery and decision-making from cryptographic authorization and policy enforcement. Every transaction is continuously validated, risk-checked, and recorded before execution.

AI-Readable Commerce Catalog — Structures merchant inventory so AI buyers can discover, understand, compare, and transact.

Autonomous Product Discovery — Converts natural-language intent into relevant product candidates and identifies the best-fit offer.

AI Growth & Ranking Engine — Optimizes product selection using intent, price, quality, availability, merchant trust, and fulfillment signals.



Merchant AI Discoverability — Improves how products are represented for machine-mediated demand, making merchants more **sellable to AI buyers**.

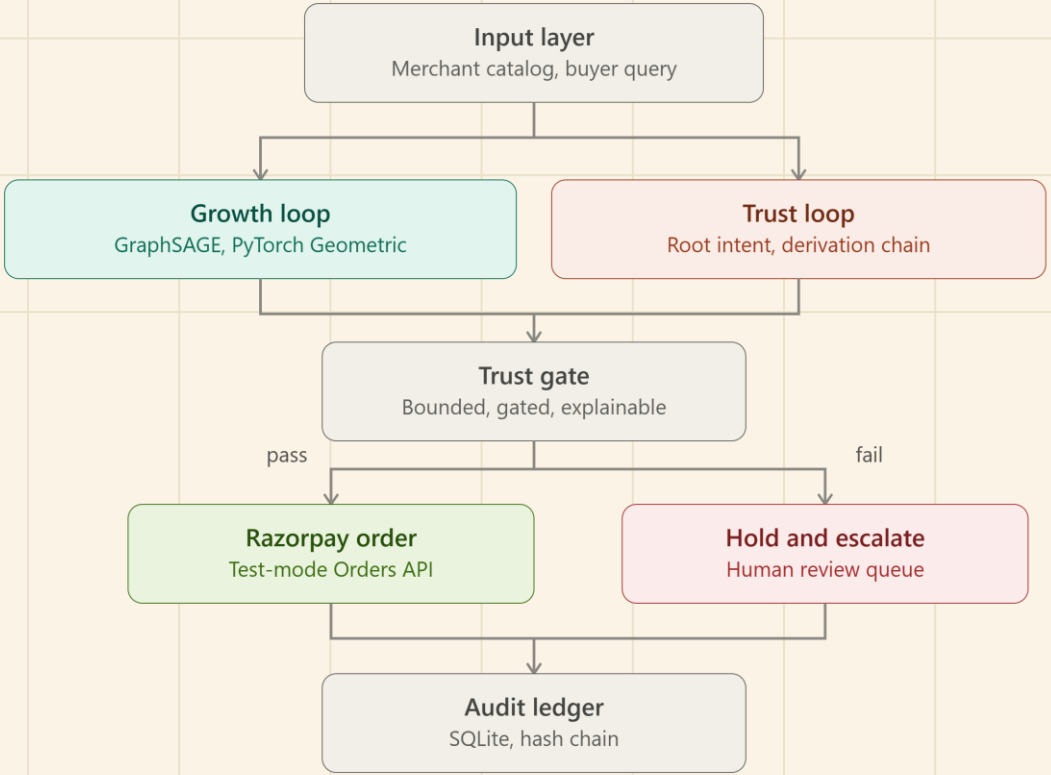
Conversational Agentic Checkout — Takes the journey from **“Find me a product”** → **“Select”** → **“Authorize”** → **“Pay.”**

Explainable Purchase Decisions — Every AI recommendation exposes its selection score and decision factors before money moves.

Bounded & Gated Transactions — Budget, merchant, category, and intent constraints are cryptographically bound and deterministically enforced.

Failure-Safe Payment + Audit Trail — Attack or policy violation → **BLOCK / ₹0 MOVED**, while successful transactions reach Razorpay Test Mode with a verifiable audit trail.

System Architecture



CLI Commands


```
# System
python backend/cli/main.py system status

# Agentic Commerce
python backend/cli/main.py agent run --intent "hotel in Goa under 3000"

# AI Growth Engine
python backend/cli/main.py growth predict --query "hotel in Goa under 3000"

# Security / Attack Lab
python backend/cli/main.py attack list
python backend/cli/main.py attack run amount-escalation
python backend/cli/main.py attack run poisoned-catalog
python backend/cli/main.py attack run recipient-substitution
python backend/cli/main.py attack run category-substitution
python backend/cli/main.py attack run derivation-tampering
python backend/cli/main.py attack run prompt-injection

# Audit / Evidence
python backend/cli/main.py ledger tail -n 5
python backend/cli/main.py ledger verify
python backend/cli/main.py ledger inspect <LEDGER_ENTRY_ID_OR_TX_ID>

# Tests
python -m pytest backend/tests/test_razorpay.py backend/tests/test_payment_gate.py -v
python -m pytest backend/tests/ -v
```

Common attacks:

Amount Escalation — Unauthorized increase in transaction value beyond the approved budget.

Poisoned Catalog — Manipulated product data attempts to influence the AI’s selection.

Recipient Substitution — Payment is redirected to an unapproved merchant or recipient.

Category Substitution — The approved product category is changed during execution.

Derivation Tampering — Authorized transaction parameters are altered after approval.

Prompt Injection — Malicious instructions attempt to override the user’s original purchase intent.

Operations/ Important Terms

1 SYSTEM ARCHITECTURE

Probabilistic Commerce Plane – responsible for intent understanding, discovery and optimization.

Deterministic Trust Boundary – responsible for deciding whether that proposed transaction is actually authorized.

2 INTENT NORMALIZATION

AEGIS converts this natural-language request into structured intent constraints:

category equals hotel,
location equals Goa,
and maximum budget equals ₹3,000.

3 AUTHORIZATION CHECKS

Is the amount within the authorized ceiling?

Is the merchant authorized?

Is the category within scope?

Is the derivation state valid?

4 CORE PRINCIPLE

AI proposes.
Policy authorizes.
Razorpay executes.

CLI Proofs


```
D:\Projects\Razorpay>python backend/cli/main.py system status
C:\Users\Sanjeev Kumar S\AppData\Roaming\Python\Python314\site-packages\torch\jit\_script.py:1485: FutureWarning: `torch
.jit.script` is not supported in Python 3.14+ and may break. Please switch to `torch.compile` or `torch.export`.
  warnings.warn(

                    AEGIS SYSTEM STATUS

    Backend             ✓
    SQLite              ✓
    Groq                ✓
    Agent Core          ✓
    Growth ML           ✓ (CPU)
    Trust Engine        ✓
    Payment Gate        ✓ (Zero Bypass)
    Razorpay            ✓ (Test Mode)
    CLI                 ✓

    Database            aegis.db
    Environment         development

    Phase               5 / 5
```

```
D:\Projects\Razorpay>python backend/cli/main.py attack run amount-escalation
C:\Users\Sanjeev Kumar S\AppData\Roaming\Python\Python314\site-packages\torch\jit\_script.py:1485: FutureWarning: `torch
.jit.script` is not supported in Python 3.14+ and may break. Please switch to `torch.compile` or `torch.export`.
  warnings.warn(

Executing Attack Scenario: amount-escalation

        Attack Simulation Result — ATK-2B35D2EB

| Security Metric          | Value                          |
|--------------------------|--------------------------------|
| Attack Status            | DETECTED                       |
| Root Intent Valid        | ✓ YES                          |
| Derivation Valid         | ✗ BROKEN / TAMPERED            |
| Verification Result      | BLOCK                          |
| Policy Decision          | BLOCK                          |
| Risk Score               | 100/100                        |
| Razorpay Order Created   | ✗ FALSE (ZERO MONEY MOVED)     |
| Ledger Entry             | LEDGER-846A0650                |

Defense Forensic Explanation: Attack 'Amount Escalation Attack' safely intercepted. Policy: BLOCK (Rule 1 Violated:
Proposed amount INR 15,000.00 exceeds authorized limit INR 3,000.00, Rule 4 Violated: Derivation chain failed
structural or semantic validation). Risk Score: 100/100 (BLOCK). Money movement prevented. Zero Razorpay orders
created.
```

```
D:\Projects\Razorpay>python backend/cli/main.py ledger tail -n 5
C:\Users\Sanjeev Kumar S\AppData\Roaming\Python\Python314\site-packages\torch\jit\_script.py:1485: FutureWarning:
.jit.script` is not supported in Python 3.14+ and may break. Please switch to `torch.compile` or `torch.export`.
  warnings.warn(
```

Aegis Append-Only Audit Ledger (Tail)

Ledger ID	Transaction ID	Outcome	Amount	Policy	Previous Hash	Current Hash
LEDGER-846A0650	PROP-ATK-0B0DAE50	ATTACK_PREVENTED	₹15,000.00	BLOCK	acd8b7ae57...	3435bcbc19...
LEDGER-B4D30B2D	PROP-ATK-D9419567	ATTACK_PREVENTED	₹15,000.00	BLOCK	582cf7722d...	acd8b7ae57...
LEDGER-7B7D75DE	PROP-ATK-D3503CD7	ATTACK_PREVENTED	₹15,000.00	BLOCK	6a874ee63a...	582cf7722d...
LEDGER-BE8DFD3E	PROP-ATK-B5E41921	ATTACK_PREVENTED	₹15,000.00	BLOCK	d3a5879faf...	6a874ee63a...
LEDGER-11057C80	AUTH-75CEC4E2	AUTHORIZED	₹2,500.00	PASS	3780182d36...	d3a5879faf...